Crosscode

What happens when one model reviews another model's code?

A first-reader guide to the tool, its data and the completed 50-task BigCodeBench-Hard experiment. Real API responses, official evaluator tests, transparent cost accounting.

Headline: Claude solo passed 45/50 tasks (90%). GPT solo passed 20/50 (40%). Claude reviewing GPT drafts reached 47/50 (94%); GPT reviewing Claude drafts reached 33/50 (66%). Review direction mattered in this sample.

Run at a glance	Observed result
Selected problems	50 of 148 hard tasks; one repetition; six workflows per task
Saved generation stages	500 total: 498 valid programs, 2 terminal API failures
Final workflow outcomes	300 attempted; 211 passes, 87 test failures, 2 API failures
Recorded API spending	$8.289765; no unresolved reservations
Highest observed pass rate	Claude self-review (AA): 48/50, or 96%
Fastest / cheapest workflow	Fastest mean: Claude solo (11.1s). Cheapest: GPT solo ($0.007962/task).

The essential qualification: AA already had 48/50 passing drafts before review and produced no pass/fail fixes. Its 96% final rate must not be described as evidence that self-review improved those drafts. The detailed before/after analysis is on page 6.

Accuracy here means passing the benchmark's supplied tests. It does not measure every requirement, code maintainability, security, or performance at repository scale. Some inspected tasks contain ambiguities between written requirements and test expectations.

Source: completed run docker-hard-fifty-fixed [S1-S4]. All dollar figures are USD. This is a 50-task pilot, not a full 148-task hard-subset result. No inference was purchased to prepare this document.

01 / PURPOSE AND WORKFLOW

What the tool does

Crosscode is a controlled experiment runner for comparing code-writing and one-pass code-review workflows. It asks two models to solve the same Python problems, optionally asks a second model to inspect each draft, and evaluates the resulting programs against the official BigCodeBench tests.

One task, six workflows

Code	Writer	Reviewer	Meaning	Calls / 50 tasks
A	Claude	None	Claude solo	1 / 50
O	GPT	None	GPT solo	1 / 50
AO	Claude	GPT	Cross-model review	2 / 100
OA	GPT	Claude	Cross-model review	2 / 100
AA	Claude	Claude	Self-review	2 / 100
OO	GPT	GPT	Self-review	2 / 100

Table 1. A = Claude Opus 5.5; O = GPT-6.1 Sol. Letters are read left to right: writer, then reviewer. The design uses ten generation stages per task, or 500 across 50 tasks. [S1, S4]

How a program moves through the system

1. Freeze the experiment

Pin dataset revisions, selected task records, model IDs, prompts, settings and evaluator images. Run every selected reference solution in Docker before paid generation.

2. Write a draft

Send only the problem statement and allowed starter interface. Ask for a complete Python program inside solution tags, plus a short confidence statement.

3. Review once, where requested

Send the problem and the actual parsed draft to a reviewer. The reviewer is told not to run or test code and returns a final program. It receives no hidden tests, canonical answer or evaluator feedback.

4. Evaluate and report

Score valid drafts and final programs in Docker. Save exact prompts/responses, programs, usage, costs, latency, test evidence and diffs. Generate tables, figures and a task audit.

Independent drafts: the Claude solo draft, the AO draft and the AA draft are separate model calls. The GPT solo, OA and OO drafts are also separate. Arrows from solo to reviewed conditions compare pipelines; only a reviewer's actual input and output establish a same-draft fix or regression.

02 / WHAT WAS TESTED

The data and experimental setup

The pinned BigCodeBench v0.1.4 snapshot contains 1,140 library-dependent Python function tasks. The separately pinned hard subset identifies 148 of those tasks. This run selects 50 hard-task IDs from a stable seed-42 ordering, with no library filters. It covers 33.8% of the hard pool, not all of BigCodeBench. [S1, S5]

Setting	Value
Task format	instruct: natural-language requirements plus required task_func interface/imports
Models requested and returned	claude-opus-5-5 and gpt-6.1-sol
Design / repetitions	paper-independent; all six conditions; one repetition
Effort / output limit	medium for both; 16,384 output tokens per call
Timeouts / retries	300 seconds API and evaluator; configured maximum retries = 0
Selection / controls	hard; seed 42; 50 tasks; $40 total / $4 per task; sequential worker
Evaluator readiness	All 50 selected canonical/reference solutions passed preflight

The tasks are practical function exercises

Examples include parsing dates from filenames, working with DataFrames, training small statistical models, drawing plots, processing audio, reading spreadsheets and extracting tables from HTML. These are function-level tasks with library dependencies, not end-to-end software projects.

Declared library coverage: pandas and numpy each appear in 26/50 tasks, matplotlib in 21, os in 14, re in 8, datetime in 7, sklearn in 6 and scipy in 4. These overlapping counts come from declared library fields; official tests can import additional dependencies. [S1]

What data goes to models, and what stays with evaluation

Models receive the task's allowed instruction text and starter code; a reviewer additionally sees its actual writer program. Hidden tests and canonical solutions remain evaluation-only. The task manifest preserves the exact selected records and content hashes. Passing preflight verifies the environment can run the reference solutions; it does not tell the models what to write.

The environment adds dependencies needed by this selection. Task 590 alone receives the authorized Wikibooks-only HTTPS transport, with a descriptive urllib User-Agent; other tasks remain offline. Tests and model programs were not rewritten. This adaptation and its image IDs are part of the recorded configuration. [S6]

03 / MAIN RESULT

Accuracy across the six workflows

Condition	Passed	Pass rate	95% bootstrap interval	Mean USD / task	Mean API seconds
A	45/50	90%	82% - 98%	0.023140	11.1
O	20/50	40%	26% - 54%	0.007962	15.0
AO	33/50	66%	52% - 80%	0.030112	21.0
OA	47/50	94%	86% - 100%	0.042998	28.8
AA	48/50	96%	90% - 100%	0.046693	19.5
OO	18/50	36%	22% - 50%	0.014890	23.3

Table 2. Reliability-inclusive results: 50 terminal attempted workflows per condition. A and O each include one API failure as a failure; their valid-program evaluation counts are 49. All reviewed final workflows were valid and evaluated. Intervals use the saved 2,000 task-cluster bootstrap resamples. [S2]

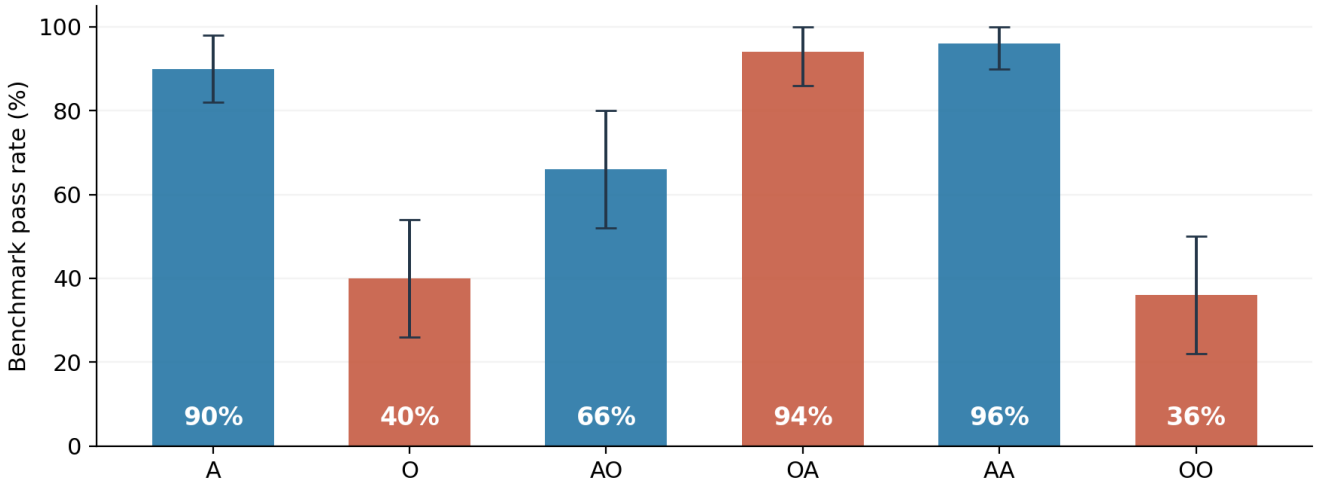


Figure 1. Pass rates with saved 95% bootstrap intervals. Blue denotes a Claude writer; orange denotes a GPT writer. These intervals describe this experimental sample and protocol, not universal model capability.

How to interpret the ranking

Claude solo: 90% pass rate at about 2.31 cents per task and 11.1 seconds mean API latency. It was both more accurate and faster on average than GPT solo in this run, while GPT solo was cheaper.

Claude as reviewer: OA reached 94%, and AA reached 96%. OA fixed many of its actual input drafts. AA's higher final rate versus solo reflects different drafts; no actual AA pass/fail fixes occurred.

GPT as reviewer: AO finished at 66%, below Claude solo; OO finished at 36%, below GPT solo. The before/after evidence shows 15 AO regressions and no OO pass/fail changes. Reasons require task-specific inspection; aggregate rates alone do not explain them.

04 / EXISTING EXPERIMENT FIGURE

Accuracy, cost and speed together

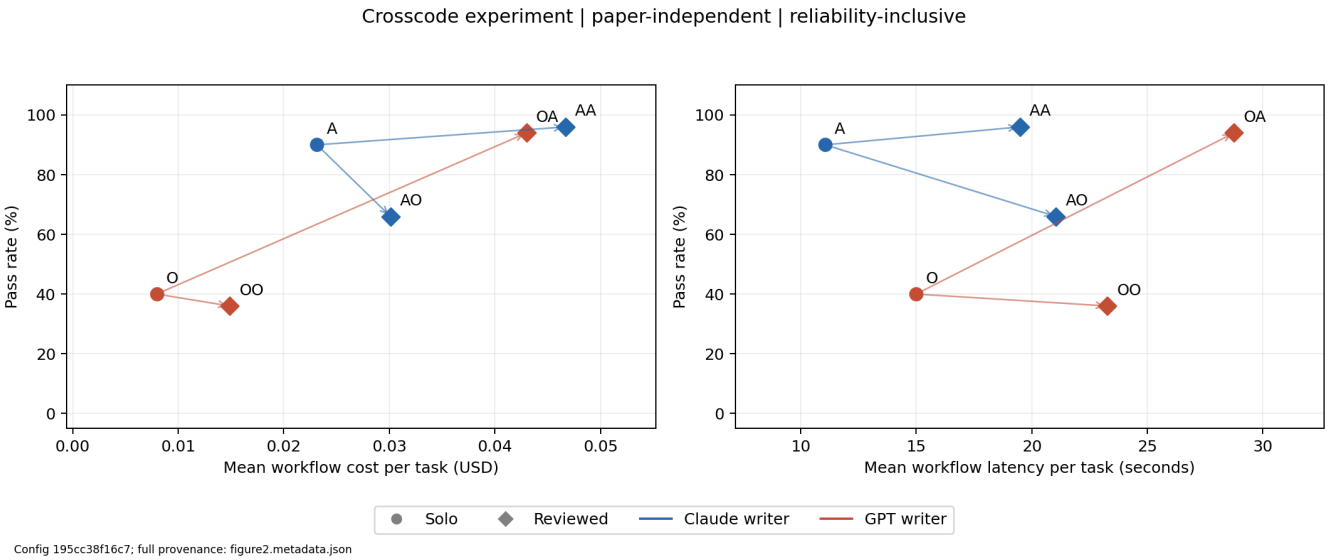


Figure 2. Original generated Figure 2, reproduced from the completed run. Left: mean workflow cost versus pass rate. Right: mean workflow API latency versus pass rate. Circles are solo workflows; diamonds include review. Colors identify the writer. Coordinates match Table 2 exactly. [S2]

Reading the figure

Higher is a larger test pass rate. Further left is cheaper in the left panel, or faster in the right panel. A workflow near the upper-left corner offers a better observed trade-off on those two axes. The arrows compare independently generated workflows; they are not before/after trajectories for the same program.

The measured trade-offs

Cost versus pass rate: GPT solo is cheapest. Claude solo buys a much higher pass rate at a higher price. OA buys another two final passes over Claude solo in this 50-task comparison, for roughly 1.99 extra cents per task. AA buys one further pass over OA, at roughly 0.37 extra cents per task. These are sample comparisons, not guaranteed improvements on another workload.

Latency versus pass rate: Claude solo is the fastest mean workflow; AA is slower but has the highest observed rate. OA is the slowest mean workflow. AA is both faster and more accurate than OA in this sample, but costs more.

Observed inefficient choices: AO costs more, takes longer and passes fewer tasks than A. OO costs more, takes longer and passes fewer than O. Those are descriptive comparisons of this run's averages, not claims about all tasks or future model versions.

A strong default from this pilot: Claude solo is the clearest measured balance of accuracy, time and cost. If reviewing an existing GPT draft, Claude review showed substantial benchmark benefit. The extra cost of routine Claude self-review is not justified by a demonstrated same-draft pass/fail improvement in this run.

The figure does not include Docker evaluation time, preflight, report rendering, user pauses or provider billing overhead. Costs are recorded token-based API charges, including known reconciled amounts, rather than the machine's electricity or a ChatGPT subscription.

05 / PIPELINE DIFFERENCES VERSUS REAL TRANSITIONS

Did the reviewer actually help?

Writer / solo rate	Claude reviewer	GPT reviewer
Claude / 90%	AA: 96% (+6 pp)	AO: 66% (-24 pp)
GPT / 40%	OA: 94% (+54 pp)	OO: 36% (-4 pp)

Table 3. Original writer-by-reviewer matrix. Deltas compare each reviewed pipeline with an independently sampled solo pipeline. pp = percentage points. [S2]

The actual before/after programs tell a different, more precise story

Review	Draft passed	Final passed	Fixes	Regressions	Kept pass / fail
AO	48/50	33/50	0	15	33 / 2
OA	22/50	47/50	25	0	22 / 3
AA	48/50	48/50	0	0	48 / 2
OO	18/50	18/50	0	0	18 / 32

Table 6. Genuine same-draft transitions, from review_transitions.csv. All 50 actual draft/final pairs per reviewed condition were scored. A fix is fail-to-pass; a regression is pass-to-fail under the official tests. [S2, S4]

AO: 48 Claude drafts passed before GPT review; only 33 final programs passed. GPT review retained 33 passes and two failures, created 15 test regressions, and fixed none. That is a 30-point decline on the actual drafts, distinct from the 24-point decline versus Claude solo.

OA: 22 GPT drafts passed initially; 47 passed after Claude review. Claude fixed 25 failures, preserved all 22 initial passes and left three failures. The actual-draft improvement is 50 points. The 54-point matrix delta includes a four-point difference in the separate GPT solo draft sample.

AA: all 48 passing drafts remained passing; both failing drafts remained failing. The +6-point matrix delta comes from the AA writer having three more passing drafts than the separate A writer. No benchmark fix or regression was observed from AA review.

OO: all 18 passing drafts and 32 failing drafts retained their outcome. Its -4-point matrix delta comes from two fewer passing initial drafts than the separate O baseline, not observed reviewer damage.

A reviewer can change code without changing its pass/fail result. Zero transitions do not mean byte-identical outputs or prove the review was useless for every unmeasured property. Conversely, a benchmark regression is not automatically a violation of the written requirements; page 10 illustrates why.

06 / HISTORICAL PRICE SNAPSHOT AND ACTUAL LEDGER

What it cost

The recorded total is **$8.289765** (about $8.29), including inherited purchases. All 500 attempts are settled; 498 are billable in the ledger and two disconnected attempts were reconciled at zero. No reservations or unknown charges remain. The $40 limit was a guardrail, not a charge. [S1, S4]

USD per 1M tokens	GPT-6.1 Sol	Claude Opus 5.5
Uncached input	2.00	4.00
Cached input read	0.10	0.20
Cache write / longer write	2.50 / 5.00	5.00 / 8.00
Output	10.00	20.00

Table 7. Frozen 24 October 2026 price snapshot stored with this run, not a current price quote. GPT write lifetimes are 30 minutes / 12 hours; Claude write lifetimes are 5 minutes / 1 hour in the runner. Output totals already include reasoning/thinking tokens. Different provider input/cache fields are normalized by the runner to avoid double charging. [S4, S7]

Provider	Saved stages	Valid responses	Recorded USD
Claude Opus 5.5	250	249	6.404008
GPT-6.1 Sol	250	249	1.885757

Each provider supplied 250 scheduled stages; one terminal disconnection occurred for each. The recorded costs are derived from attempts, not added a second time from the original run. [S4]

Valid-response usage: Claude returned 184,627 input and 283,275 output tokens, with no cache reads/writes. GPT returned 113,675 input and 165,402 output tokens; 8,773 input tokens were cache writes, with no cache reads. Output totals include reasoning; they are not all visible program text. [S4]

Workflow	Writer USD	Review USD	Total USD	USD / passed final
A	1.1570	0.0000	1.1570	0.0257
O	0.3981	0.0000	0.3981	0.0199
AO	1.1756	0.3300	1.5056	0.0456
OA	0.4131	1.7368	2.1499	0.0457
AA	1.1719	1.1628	2.3347	0.0486
OO	0.3936	0.3509	0.7445	0.0414

Table 8. Actual workflow spending across 50 tasks per condition. USD/passed final = total condition spending divided by its passing finals; this is a retrospective efficiency ratio, not the expected price of solving an arbitrary new task. All 50 cost rows per condition are known. Totals across conditions sum to the ledger (up to rounding). [S2, S4]

Cost is calculated from the usage returned by each provider using the frozen price snapshot; verified reconciliation handles missing usage. The ledger is an experiment accounting record, not a substitute for vendor invoices, tax or currency conversion. No current vendor prices were fetched for this report.

07 / API LATENCY, NOT ELAPSED BENCHMARK RUNTIME

How long the workflows took

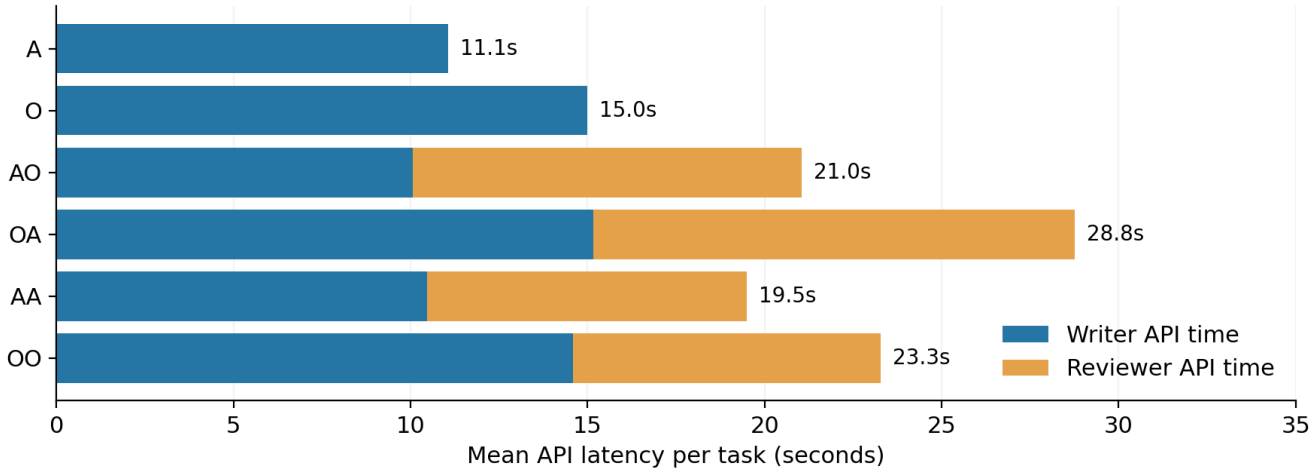


Figure 3. Mean writer and reviewer API time per workflow, stacked. Values are recomputed from the saved all_metrics.csv. The two terminal disconnections contribute their measured API time; model review is a sequential second call. [S2-S4]

Condition	Mean seconds	Median seconds	90th percentile seconds
A	11.1	10.1	14.5
O	15.0	14.0	24.8
AO	21.0	19.1	31.4
OA	28.8	26.8	41.2
AA	19.5	18.5	26.6
OO	23.3	20.9	34.4

New descriptive latency summary from the 50 saved workflow durations per condition; percentile uses linear interpolation. These are observed durations, not service-level guarantees. [S3]

Claude solo averaged 11.1 seconds per task versus 15.0 seconds for GPT solo. AA averaged 19.5 seconds, AO 21.0, OO 23.3 and OA 28.8. Provider speed can depend on output length, reasoning behavior, account routing and load; this run does not isolate which factor caused the differences.

Across all six workflows and 50 tasks, summed recorded API time is 98.8 minutes. This sums work, not an uninterrupted end-to-end runtime. Saved call timestamps span 25 Oct 22:39 to 26 Oct 02:29 Asia/Karachi; that interval includes pauses and other phases and must not be treated as generation time.

Docker setup, canonical preflight, evaluation, report building and user interruptions are outside the plotted API latency. The two pre-resume network failures were retained rather than silently retried. No time-to-first-token or generated-program runtime benchmark is reported.

08 / PAIRED STATISTICS AND SENSITIVITY

How strong is the evidence?

The unit of comparison is the same selected task under two workflows. Exact two-sided McNemar tests use only tasks where the two outcomes differ. The runner adjusts across all 15 pairwise comparisons using Benjamini-Hochberg (BH), separately within each population and repetition. The six original displayed comparisons follow. [S2]

Comparison	First only	Second only	Exact p	BH adjusted p
OA versus O	27	0	1.49e-08	5.59e-08
OO versus O	0	2	0.5	0.577
AO versus A	1	13	0.00183	0.0025
AA versus A	3	0	0.25	0.312
AA versus AO	15	0	6.1e-05	0.000131
AO versus OA	0	14	0.000122	0.000229

Table 4. All rows have 50 paired tasks; the adjustment family has 15 comparisons. First-only means the first named condition passes and the second fails, and vice versa. These are pipeline comparisons, not necessarily actual reviewer fixes. [S2]

Within this sample, OA versus O and AO versus A show strong paired differences after adjustment. AA versus A (adjusted p = 0.3125) and OO versus O (0.5769) do not provide clear evidence of a pipeline difference. A non-small p-value does not establish equality; the sample has only 50 tasks and one repetition.

Does excluding the API failures change the picture?

Condition	Reliability-inclusive / 50	Complete-case / 48
A	45/50 (90%)	44/48 (91.7%)
O	20/50 (40%)	19/48 (39.6%)
AO	33/50 (66%)	31/48 (64.6%)
OA	47/50 (94%)	45/48 (93.8%)
AA	48/50 (96%)	46/48 (95.8%)
OO	18/50 (36%)	17/48 (35.4%)

Table 9. Complete-case analysis drops tasks 108 and 915 from every condition because their solo workflows had API failures. Its smaller paired population is 48 tasks. Reliability-inclusive analysis counts those terminal API errors as failures in their affected workflows. [S2]

The qualitative pattern is similar in both populations. This report leads with reliability-inclusive results because they include the actual operational failures. Bootstrap intervals use 2,000 resamples of whole task clusters, seed 42; they do not capture all uncertainty from future tasks, model updates, prompt choice or repeated sampling.

09 / TABLE 5: INSPECTED TRANSITIONS AND BENCHMARK CAVEATS

What went right or wrong in examples

The original generated Table 5 selects the first two task IDs per condition and true transition. Its causal descriptions were marked pending inspection. For this report, the following examples were checked against saved code diffs, supplied instructions, official tests and evaluator tracebacks. These observations explain only these cases; the run's annotation files were not changed. [S1, S4]

Task / workflow	Measured transition	Evidence-based observation
1057 / OA	Fail -> pass	Claude changed missing/default input handling and the default animal/food lists. The GPT draft returned a 0 x 0 frame for no arguments and 3 x 3 for a partial-default test; tests expected 10 x 7 and 3 x 7. The final program passed.
1057 / AO	Pass -> fail	GPT normalized omitted arguments to empty lists and returned an empty DataFrame when both were absent. The official default-input test expected 10 x 7. The written instruction says both empty or not provided should return empty, creating a specification/test tension.
1008 / AO	Pass -> fail	GPT wrapped a requests ConnectionError as HTTPError. The supplied prose asks for HTTPError on request failures, but test_network_error expects the original ConnectionError to propagate. The traceback identifies this exception-type mismatch.

Table 5. Task IDs are BigCodeBench IDs. The first example is a measured benchmark repair; the latter two are measured test regressions with instruction/test ambiguities. No tests were modified, excluded or supplied to the models.

Why these examples matter

A score change records agreement with this test suite. It does not by itself establish that the reviewer made the program less faithful to the natural-language request. In both inspected AO examples, the changed behavior can be read as aligning with part of the instruction while failing the official test. Calling all 15 AO regressions careless code review would go beyond the evidence.

Task 1057 also leaves exact predefined lists underspecified in the shown instruction. Successful code supplies values matching the benchmark's expectations; this report cannot establish how a model obtained them. Benchmark contamination or memorization is a possibility, not a demonstrated cause.

The generated selection also contains task 187 as an OA fix. Its outcome is verified in the saved evaluator evidence, but a full causal explanation is not asserted here. Exact model responses, parsed code, diffs and test details remain available in the task audit for every workflow.

Interpret the headline as **benchmark test performance under this recorded protocol**. A requirements audit or independent test suite is needed before translating these figures into production-quality claims.

10 / WHAT WAS ACTUALLY FINISHED

Completion, reliability and visibility

Layer	Completion and outcomes
Problem selection	50 tasks x 6 final workflows = 300 attempted final workflows
Generation	500 saved stages: 498 valid programs and 2 terminal API failures
Program evaluation	498 valid programs evaluated: 347 pass, 151 fail, zero infrastructure failures
Final workflow evaluation	298 valid finals: 211 pass, 87 fail; 2 API failures complete the 300 workflows
Actual review draft evaluation	200 drafts from AO, OA, AA and OO were also evaluated; these are not extra final workflows
Accounting	500 settled attempts; 498 billable attempts; zero unresolved/reserved USD

Stage-level 347/498 must not be presented as the final workflow accuracy. It includes intermediate drafts. Likewise, 500 saved stages means execution is complete, not that every task passed. [S1-S4]

The two operational failures

Claude solo on task 108 and GPT solo on task 915 recorded RemoteDisconnected API failures. Their responses are unavailable, and they were not silently regenerated. Billing was reconciled at zero; they are counted as failures in reliability-inclusive results. They are communication failures rather than evaluated incorrect programs.

What a user can inspect

The task audit records each task's full requirements and interface, every requested workflow and exact saved writer/reviewer text, prompts, parsed programs, provider response structures, usage, prices, elapsed time, attempts, diffs and evaluator evidence. It distinguishes unstarted, invalid, failed and passed work. Provider-private reasoning that was not returned cannot be recovered.

Live PowerShell progress identifies task, condition, role, model and attempt ID, reports cache reuse and spending, and emits an elapsed-time heartbeat while waiting. Events are appended to progress.log. The log is for operational visibility; the audit provides the full program-level evidence.

How progress and money are protected

Saved stage identities bind the exact task, prompt, model/settings and parent draft. SQLite checkpoints preserve completed work. Conservative atomic reservations check total and per-task budgets before dispatch; unknown usage blocks further generation until reconciliation. Resuming compatible work reuses it. Limits are local guardrails, not provider-enforced invoice caps.

11 / SCOPE AND PRACTICAL INTERPRETATION

What to conclude, and what to test next

What this experiment supports

For this 50-task sample, Claude solo delivered a much higher test pass rate than GPT solo, with lower mean API latency but higher cost. Claude review substantially improved its actual GPT input drafts. GPT review damaged test outcomes on some otherwise passing Claude drafts. Review direction therefore matters, and a second model call is not automatically beneficial.

Claude self-review produced the highest observed final rate, but the same drafts had already achieved that rate. Its extra cost and latency purchased no measured fail-to-pass transitions in this run. A direct comparison with the separate solo pipeline cannot isolate self-review's causal effect.

What it does not support

This is not a general ranking of these model families. It uses 50 selected tasks from 148 hard tasks, one repetition, one prompt pair, one effort setting and a 16,384-token ceiling. Equal effort labels and caps do not equalize compute or reasoning. Model aliases can change; exact requested and returned identifiers are saved, but not all serving details are exposed.

These tasks are library-dependent functions. They do not validate repository understanding, multi-file changes, deployment, security, human review utility or long-term maintenance. Test coverage and instruction/test disagreement limit what a pass means. Contamination may affect results. Evaluator dependency versions and the live Wikibooks page add reproducibility constraints.

A sensible next experiment

First inspect the remaining review regressions and failures and tag requirements/test disagreements separately from clear code defects. Then repeat generation on the same tasks to quantify sampling variation, while keeping prompts and evaluator settings fixed. To estimate review benefit directly, compare each reviewer's actual draft with its final program, rather than attributing independent-draft baseline differences to review.

A broader evaluation could cover all 148 hard tasks after their canonical preflight passes, and add an independent test set or requirements audit. Dependency/network support for the other 98 tasks is not established by this run. Keep those changes as new configurations and disclose which model outputs were reused.

Terms used in this report

Term	Meaning
Pass rate / accuracy	Fraction of workflow outcomes passing the supplied official tests
pp / SID	Percentage-point difference from a solo baseline; a pipeline comparison
Fix / regression	The same actual reviewer input changes from fail to pass / pass to fail
Complete-case	Only tasks evaluated in every requested condition (48 here)
Reliability-inclusive	Includes terminal invalid/API attempts as failures; excludes infrastructure-only and unstarted work

APPENDIX A / SELECTED TASKS AND FINAL OUTCOMES

Task-level results (1-25 of 50)

P = official tests passed; F = valid program failed tests; E = terminal API failure. No infrastructure failures occurred. Short task descriptions below are editorial summaries of the frozen instructions, not substitutes for their full requirements. [S1, S3]

ID	Task requirement (abridged)	A	O	AO	OA	AA	OO
969	Computes the MinMax-normalized cumulative sum for each numeric column in the given...	P	P	P	P	P	P
108	Decomposes a time series in the 'value' column of a DataFrame into trend, seasonality,...	E	P	P	P	P	P
526	Read a list of dictionaries from a JSON file, calculate the mean and median for each key...	P	P	P	P	P	P
915	Identifies and plots outliers in the 'closing_price' column of a given DataFrame using...	P	E	P	P	P	F
917	Forecasts the share closing prices for the next 7 days using the ARIMA model and plots...	P	F	P	P	P	F
914	Predicts the stock closing prices for the next 7 days using simple linear regression and...	P	F	P	P	P	F
92	Perform K-means clustering on a dataset and generate a scatter plot visualizing the...	P	P	P	P	P	P
15	Execute a list of shell commands read from a CSV file and save the outputs in separate...	P	F	P	P	P	F
945	Generates a time series of sales data starting from a specified date, then use linear...	F	F	F	F	F	F
870	Calculate the mean of numerical values in each position across tuples in a list....	P	P	P	P	P	P
574	Create a noisy sine wave of a specified length and adjusts a curve using curve_fit from...	P	F	F	P	P	F
590	Extracts the text and href attributes of all anchor tags from a given URL's HTML content,...	F	F	F	P	P	F
532	Identify and count duplicate values in a DataFrame's 'value' column. This function also...	P	F	F	P	P	F
211	Download and keep a zip file from a URL, extract its contents to the specified directory,...	P	F	F	P	P	F
208	Generate and draw a random sequence of "elements" number of steps. The steps are either...	P	P	P	P	P	P
162	Analyzes and visualizes the distribution of word lengths in a text. The function...	P	P	P	P	P	P
511	Analyze a list of employee data and calculate statistics for a given column. If the data...	P	F	P	P	P	F
227	Creates an MxN matrix from a list L, normalizes it based on the sound pressure level...	P	F	F	P	P	F
678	Processes JSON files in a directory. The function reads each JSON file alphabetically...	P	P	P	P	P	P
553	Generate a pandas DataFrame with random values based on lists 'a' and 'b', and plot it as...	P	P	P	P	P	P
657	Generate word vectors from a list of texts using the gensim Word2Vec model and...	P	P	F	P	P	P
399	Create a diagram of a sine wave and cosine wave with a given frequency and return the...	P	F	P	P	P	F
988	Evaluates each item (files and directories) in a given directory against specified...	P	F	F	P	P	F
1003	Fetches and parses an XML file from a specified URL, then converts it into a Pandas...	P	P	P	P	P	P
845	Calculate the similarity values between two texts based on the cosine similarity and the...	P	P	P	P	P	P

Selection order is the original frozen seed-42 prefix. All 50 reference solutions passed; P/F/E above refer to model workflows, not reference preflight. Full exact requirements and programs are in report/task-audit/index.md within the run.

APPENDIX A / SELECTED TASKS AND FINAL OUTCOMES

Task-level results (26-50 of 50)

P = official tests passed; F = valid program failed tests; E = terminal API failure. No infrastructure failures occurred. Short task descriptions below are editorial summaries of the frozen instructions, not substitutes for their full requirements. [S1, S3]

ID	Task requirement (abridged)	A	O	AO	OA	AA	OO
826	Move files from the source directory to the target directory based on a specified...	P	P	P	P	P	F
906	Archives all processed files from a source directory to a target directory. The function...	P	P	P	P	P	P
857	Transfer files from one directory (SOURCE_DIR) to another (DEST_DIR) based on the...	P	F	F	P	P	F
502	Generates a graph of daily activity durations for a specified number of days in the past...	P	F	P	P	P	F
952	Randomly assigns a specified number of tasks to employees with a due date of the current...	P	F	P	P	P	F
760	Creates a random DataFrame with 100 records. Each record consists of an ID (ranging from...	P	P	P	P	P	P
99	Draws a seaborn pair plot of the iris dataset using Arial font. This function sets the...	P	P	P	P	P	P
777	Unzip all zip files in a directory whose name matches a certain pattern by splitting the...	F	F	F	P	P	F
199	Generate a weather report for specified cities at a given UTC datetime.	P	P	P	P	P	P
401	Initializes a Flask-Mail instance for sending emails using the generated Flask...	P	P	P	P	P	P
582	Create a list of normally distributed random numbers and plot their histogram and...	P	F	P	P	P	F
594	Generate weather data for the specified number of hours, save it in a CSV file with...	P	F	F	P	P	F
560	This function plots a bar chart of monthly data values for a single year, with 'month' on...	P	F	P	P	P	F
1057	Create a DataFrame with combinations of animals and foods in a 'animal:food' format....	P	F	F	P	P	F
454	Moves files with a specified extension from a source directory to a destination...	P	F	F	P	P	F
513	Analyze a list of fitness data, calculate the sum, the mean, the minimum, the maximum of...	P	F	P	P	P	F
368	Moves a random file from the source directory to the specified destination directory.	P	P	P	P	P	F
443	Calculate the product of a matrix 'P' and a 3D tensor 'T', flatten the result, apply...	P	F	P	P	P	F
214	Generate a random RGB image and view it.	P	F	F	P	P	F
409	Calculate the mean, median, and standard deviation of the data from a specific column in...	F	F	F	F	F	F
942	Create and visualize a sales report for different categories over a period of time.	P	F	P	P	P	F
477	Create a DataFrame with a given number of rows (N) and 3 columns: "x" and "y" with random...	P	P	P	P	P	P
187	Create a GeoPandas DataFrame for a list of cities with randomly generated coordinates...	P	F	F	P	P	F
763	Read a list of dictionaries from a JSON file, calculate the mean and median for each key,...	P	F	P	P	P	F
1008	Extracts and converts data from a specified HTML table based on the given 'table_id' on a...	P	F	F	F	P	F

Selection order is the original frozen seed-42 prefix. All 50 reference solutions passed; P/F/E above refer to model workflows, not reference preflight. Full exact requirements and programs are in report/task-audit/index.md within the run.

APPENDIX B / AUDIT TRAIL

Sources, reproducibility and file guide

This report was compiled from saved local evidence on 26 October 2026. It makes no new provider calls and does not change checkpoints, model outputs, annotations or evaluation results. Historical pricing is quoted from the run, without asserting current vendor tariffs.

[S1] Frozen run configuration and manifest. runs/docker-hard-fifty-fixed/config.json; manifest.json. Exact task fields, selection, model settings and hashes.

[S2] Generated population tables and figure. report/reliability-inclusive/{table2,table3,table4,table5,all_metrics,review_transitions}.csv; figure2.png. Sensitivity: report/complete-case/table2.csv.

[S3] Saved workflow analysis. report/results.json, including metadata, cumulative ledger and 300 final workflow records. Supplementary latency percentiles in this PDF are calculated from these durations.

[S4] Original attempt and program evidence. checkpoint.sqlite: stages, attempts and evaluations; report/task-audit/index.md. Read-only queries verified totals, outcomes and code examples for this report.

[S5] Pinned benchmark and study sources. sources.lock.json. BigCodeBench full snapshot b74c0d0bf70d2c0bc459be537895cca163007f1a; hard snapshot 298d2cc7b96612e15e47313c3603ee124cee0c1f; v0.1.4.

[S6] Evaluator and continuation provenance. docker/wikibooks.image.lock.json; docs/EVALUATION.md; preflight.json; migration.json. Original 100 generation stages were inherited and saved programs rescored under the new evaluator.

[S7] Frozen pricing and accounting implementation. Per-stage pricing snapshot dated 2026-10-24, plus crosscode/providers.py and crosscode/store.py. Documentation links recorded with that snapshot are listed below.

All relative paths above are under the CrossCode workspace unless they start with report/ or name a run-local file. Run-local files are under runs/docker-hard-fifty-fixed/.

Primary project and data links

BigCodeBench source

Full dataset

Hard subset

Cross-model review reference implementation

GPT pricing documentation recorded by the run

Claude pricing documentation recorded by the run

Immutable configuration identity

195cc38f16c75479932b7b7a4e9445b9
20c88864b898547e1fcbcb06dc19919f

Evaluator: afd0ca41f44b14dd7460a52cde246c49e0e4fff6dd84256e2643728917072c7f
Proxy: 3304f6c216ff7e417f71a675773d5a1c939dfa4414423fb009e5404685a19ef9

Image values above are SHA256 IDs. Evaluator source commit: 09dd993f46c3fbf3a799465bb96d524edcb0b199. The continuation includes inherited purchases from docker-hard-five-check; do not add their overlapping ledgers. The associated .data.json file preserves the numerical report inputs and derived summaries.